SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	***Course Title:***	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	***Assessment:***	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	***Total Marks:***	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	N. Adi Narayana	**Reg. No.:**	192525346
Section / Batch:	AIML	**Date:**	17/07/2026

Code of Conduct

I N. Adi Narayana *(Name / Reg No)* *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.* Signature of the Student: N. Adi

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:		**100 Marks**

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Pictorially insert an element at 3rd position in a doubly linked list. Having 6 elements in the list and delete the element at 2nd position from the list. implement and analyse:

Doubly linked list

Assume the 6 elements are:

10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60

In a doubly linked list, inserting at a given position means updating both next and prev links around the new node, and deleting at a given position means bypassing the node and reconnecting its neighbors.

Insert at 3rd position

"3rd position" means the new node becomes the 3rd node in the list, so it is inserted between 2nd & 3rd node

Before:

10 <-> 20 <-> 30 <-> 40 <-> 50 <-> 60

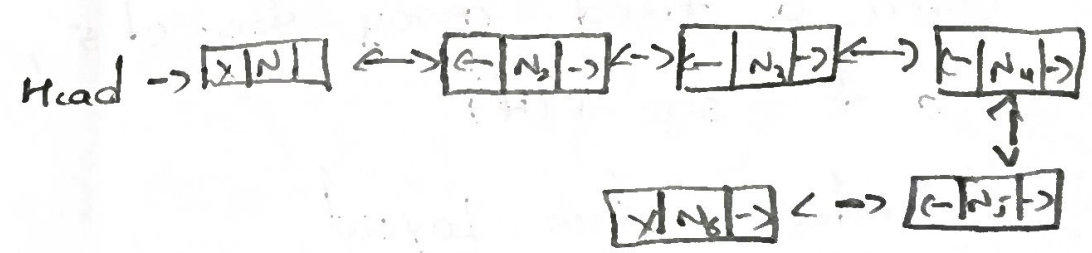
After: 10 <-> 20 <-> ~~25~~ <-> 30 <-> 40 <-> 50 <-> 60

20.next = 25

25.prev = 20

25.next = 30

30.prev = 25



Delete at 2nd position:

Now delete the node at position 2

Before:

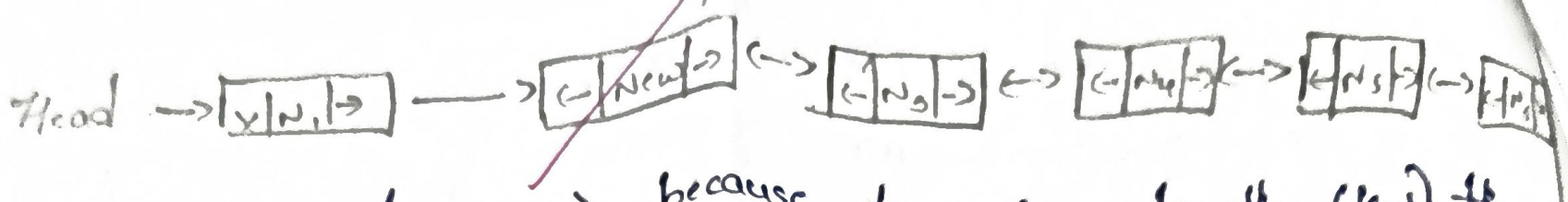
10 <-> 20 <-> 25 <-> 30 <-> 40 <-> 50 <-> 60

After:

10 <-> 25 <-> 30 -> 40 -> 50 -> 60

10.next = 25

25.prev = 20



Time complexity : O(k) because traversing to the (k-1) th. Position scales linearly with the index. The pointer rewires executes in constant time.

② Implement Binary operations using stack. Assume the size of the stack is 5 and have a value of 22, 55, 33, 66, 88 in the stack from 0 position to size-1. Now perform the following operat

1) Invert 2) POP [] 3) Pop [] 3) Pop[] 4) Push [90] 5) Push [36] 6) Push [11] 7) Push [88] 8) POP [] 9) POP[]

Ans: Given a fixed array size of 5, the stack positions, index from 0 to size-1 (4)

Initial Stack Layout

* Bottom to Top : 22, 55, 33, 66, 88

* top tracking index points to position 4

text

Index [4] → 88 ← Top

Index [3] → 66

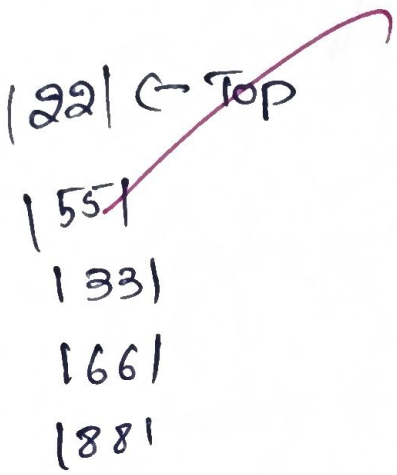
Index [2] → 33

Index [1] → 55

Index [0] → 22

1) Invert the elements in the stack

the relative ordering Index 0 becomes the top index 4 becomes



Pop()

* Removes 22
* stack state :[88, 66, 33, 55) Top is 55

Pop()

* Removes 55
* stack state : [88, 66, 33) Top is 33)

Pop()

* Removes 33.
* stack state : [88, 66] Top is 66)

5) Push(90)

- Adds 90
- stack

6) Push(36)

- Adds 36.
- stack state [88, 66, 90, 36) (top is 36)

7) Push [11]

- Adds 11. stack reaches max capacity
- stack state.

[88, 66, 90, 36, 11]

8) Push [88]

- Overflow Error

The stack has a fixed size of 5 and is currently full. This operation is rejected and fails to alter the stack

a) POP()

- Removes 11.
- state state : [88, 66, 90] (top is 90)

(2) pop()

→ Removes 36.

* stack state : [88, 66, 90)

⇒ final elements : [88, 66, 90]

⇒ Top pointer position :

Index 2 (Value : 90)